ROS 2 URDF & Xacro

Beginner reference notes: RX200 wrist-mounted RGB camera

Learning goal: Understand each part of a camera Xacro, how the camera attaches to an arm, how TF frames work, and what must be added for Gazebo Fortress to publish images.

Starting point: The provided wrist camera XML defines a physical model and frame tree; it does **not** yet define an image-producing sensor.

Contents

1. URDF, Xacro and robot links

2. Arguments and properties

3. Camera geometry and physics

4. Fixed mounting joint

5. Optical frame and TF

6. Gazebo Fortress RGB sensor

7. ROS 2 image pipeline and tests

8. Beginner checklist and common errors

Example naming assumes `robot_name=rx200`. Always inspect your generated URDF because actual Interbotix link names may differ.

Chapter 1 - URDF, Xacro and robot links

URDF describes a robot as **links** (rigid bodies) connected by **joints**. Each link may include visual, collision and inertial information. **Xacro** adds reusable parameters, expressions and macros to generate URDF XML.

```
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">
  <link name="camera_link"/>
  <joint name="camera_mount" type="fixed">
    <parent link="ee_gripper_link"/>
    <child link="camera_link"/>
  </joint>
</robot>
```

Remember: a link name is an exact identifier. A slash in `rx200/wrist_camera_link` is part of the name, not an automatically created ROS namespace.

Chapter 2 - Arguments and properties

```
<xacro:arg name="robot_name" default="rx200"/>
<xacro:arg name="wrist_camera_z" default="0.045"/>
<xacro:property name="camera_width" value="0.035"/>
<link name="$(arg robot_name)/wrist_camera_link"/>
<box size="${camera_width} 0.025 0.025"/>
```

$(arg robot_name) reads a declared Xacro argument; **${camera_width}** evaluates a Xacro expression/property. With the sample defaults, the link becomes `rx200/wrist_camera_link`.

The supplied camera snippet uses `$(arg robot_name)` but does not declare it locally. Make sure the enclosing Xacro declares/provides it before expansion.

Mounting offsets: measured in metres and radians

Argument	Default	Meaning
wrist_camera_x	-0.015 m	1.5 cm in negative parent X
wrist_camera_y	0 m	No parent Y offset
wrist_camera_z	0.045 m	4.5 cm in positive parent Z
roll, pitch, yaw	0 rad each	No rotation relative to parent link

These offsets are measured in the **parent link frame**, not directly in world coordinates. Check the actual `ee_gripper_link` axes before deciding whether the lens faces the fingers.

Chapter 3 - Camera geometry and physics

The example treats the camera as a uniform rectangular box. This is useful for visualization and basic contact simulation, but is not a detailed housing or lens model.

URDF element	What it does
visual	Black 35 x 25 x 25 mm box rendered in RViz/Gazebo.
collision	Same-size box used in contact calculations.
inertial	30 g mass and box moments of inertia.

```
<visual>
```

```
  <geometry><box size="${camera_width}
    ${camera_height} ${camera_depth}"/></geometry>
</visual>
<collision> ... same box ... </collision>
<inertial><mass value="${camera_mass}"/> ... </inertial>
```

For a uniform box: Ixx = m(h²+d²)/12; Iyy = m(w²+d²)/12; Izz = m(w²+h²)/12. This model uses approximately Ixx=3.125e-6 and Iyy=Izz=4.625e-6 kg m². The camera mount and cable mass are not represented.

Chapter 4 - Fixed mounting joint

```
<joint name="$(arg robot_name)/wrist_camera_joint"
       type="fixed">
  <parent link="$(arg robot_name)/ee_gripper_link"/>
  <child link="$(arg robot_name)/wrist_camera_link"/>
  <origin xyz="-0.015 0 0.045" rpy="0 0 0"/>
</joint>
```

A **fixed joint** makes the camera move rigidly with the gripper. The constant transform from gripper to camera is translation (-0.015, 0, 0.045) m and zero relative rotation.

```
base_link
   ... arm joints ...
      ee_gripper_link
         |
         +-- [fixed] wrist_camera_link
                 |
                 +-- [fixed] wrist_camera_optical_frame
```

Key equation: world_T_camera = world_T_gripper x gripper_T_camera. TF2 uses chains like this to locate the camera while the arm moves.

Chapter 5 - Optical frame and TF

A camera optical frame uses the ROS optical convention: **+Z forward (into image scene), +X right, +Y down**. A common robot-body convention is +X forward, +Y left, +Z up.

```
<link name="$(arg robot_name)/wrist_camera_optical_frame"/>
<joint name="$(arg robot_name)/wrist_camera_optical_joint"
       type="fixed">
  <parent link="$(arg robot_name)/wrist_camera_link"/>
  <child link="$(arg robot_name)/wrist_camera_optical_frame"/>
  <origin xyz="0 0 0" rpy="-1.57079632679 0 -1.57079632679"/>
</joint>
```

This standard rotation makes optical +Z align with camera-body +X. Because camera body rotation relative to gripper is zero in the given code, the optical viewing direction aligns with **gripper +X**. Verify that this is actually toward the object.

From image pixel to camera-space point

Given metric depth Z and camera intrinsics fx, fy, cx, cy: **X = (u-cx) Z / fx**, **Y = (v-cy) Z / fy**. Then transform the point from the optical frame to the robot base frame. **RGB alone does not directly provide Z.**

Chapter 6 - Add an RGB sensor for Gazebo Fortress

Visual/collision links are not cameras. Gazebo requires a **camera sensor**, a loaded **Sensors system**, and a bridge if ROS 2 should receive its images. This is an illustrative sensor block for a Fortress-compatible URDF/Xacro:

```
<gazebo reference="$(arg robot_name)/wrist_camera_link">
  <sensor name="wrist_rgb" type="camera">
    <always_on>true</always_on>
    <update_rate>30</update_rate>
    <topic>/wrist_camera/image_raw</topic>
    <camera>
      <horizontal_fov>1.047</horizontal_fov>
      <image><width>640</width><height>480</height>
        <format>R8G8B8</format></image>
      <clip><near>0.01</near><far>5.0</far></clip>
    </camera>
  </sensor>
</gazebo>
```

Frame warning: setting a URDF optical link does not automatically configure the Gazebo sensor image header. Ensure sensor pose and advertised frame match the actual optical frame. Gazebo sensor axis conventions and URDF conversion should be checked using a known target.

Chapter 7 - ROS 2 image pipeline and testing

```
Gazebo camera sensor
      |  Gazebo Transport image topic
      v
ros_gz_bridge (Fortress / Ignition message type)
      |  sensor_msgs/msg/Image
      v
ROS 2 image topic -> perception / visualization
```

With Fortress, the world also needs the matching `ignition-gazebo-sensors-system` plugin (typically using OGRE2). Topic names and message type names depend on the installed bridge version.

Suggested validation commands

```
# Expand Xacro to URDF (adapt the input path)
ros2 run xacro xacro robot.urdf.xacro > /tmp/robot.urdf

# Check model structure
check_urdf /tmp/robot.urdf

# Inspect names in the generated file
grep 'wrist_camera' /tmp/robot.urdf

# Gazebo Fortress transport topics
ign topic -l

# ROS 2 topics, once bridged
ros2 topic list
ros2 topic hz /wrist_camera/image_raw

# Once robot_state_publisher is publishing TF:
ros2 run tf2_ros tf2_echo   rx200/ee_gripper_link rx200/wrist_camera_optical_frame
```

Example Fortress bridge command (verify package/version-specific syntax):

```
ros2 run ros_gz_bridge parameter_bridge   '/wrist_camera/image_raw@sensor_msgs/msg/Image[ignition.msgs.Image'
```

If your installation is older, the bridge executable or Gazebo message naming may differ. Use `ros2 pkg list | grep -E "ros_gz|ros_ign"` to check what is installed.

Chapter 8 - Checklist and common errors

• The Xacro input declares or receives robot_name.

• Parent and child link names match exactly after Xacro expansion.

• Camera collision box does not intersect gripper geometry unexpectedly.

• Optical +Z points toward the intended scene.

• Camera sensor is defined and the Fortress Sensors system loads.

• Gazebo image topic is published and mapped to ROS 2 by a bridge.

• TF tree includes optical frame; image frame_id is consistent with it.

• Do not treat 2D RGB pixels as metric 3D points without depth or a scale-estimation method.

Quick troubleshooting

Symptom	Likely reason / check
Xacro arg missing	Declare <xacro:arg name="robot_name" .../> in scope.
Invalid parent link	Inspect generated URDF; check slash/prefix.

Symptom	Likely reason / check
Box appears but no image	Missing sensor, Sensors system, or bridge.
Image exists but wrong aim	Check joint rpy, sensor pose and optical convention.
No TF frame	Check robot_state_publisher and joint state publishing.

Recommended file separation: keep camera links/joints in `rx200_wrist_camera.urdf.xacro` and simulation-only sensor definitions in `rx200_wrist_camera.gazebo.xacro`. Include both from the assembled robot Xacro only when simulation is enabled.

Official further reading: ROS 2 URDF tutorials (docs.ros.org); Xacro wiki (wiki.ros.org/xacro); Gazebo Fortress sensor and ROS bridge documentation (gazebosim.org/docs/fortress).